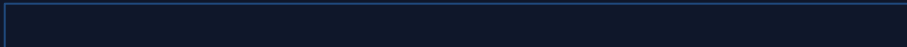


MINICURSO

Python Básico para Datos



Ambientes de Desenvolvimento Python

VS Code

Leve, extremamente customizável e o verdadeiro "canivete suíço" do programador moderno.

Requer a instalação de extensões essenciais como Pylance e suporte ao Jupyter para otimizar o fluxo de desenvolvimento Python.

PyCharm

Uma IDE robusta, completa e pronta para uso profissional ("out-of-the-box").

Especializada em Python, oferece de forma nativa ferramentas avançadas de refatoração de código, depuração inteligente e integração com frameworks web.

Jupyter Notebook

Padrão da indústria para computação interativa e exploração rápida de dados.

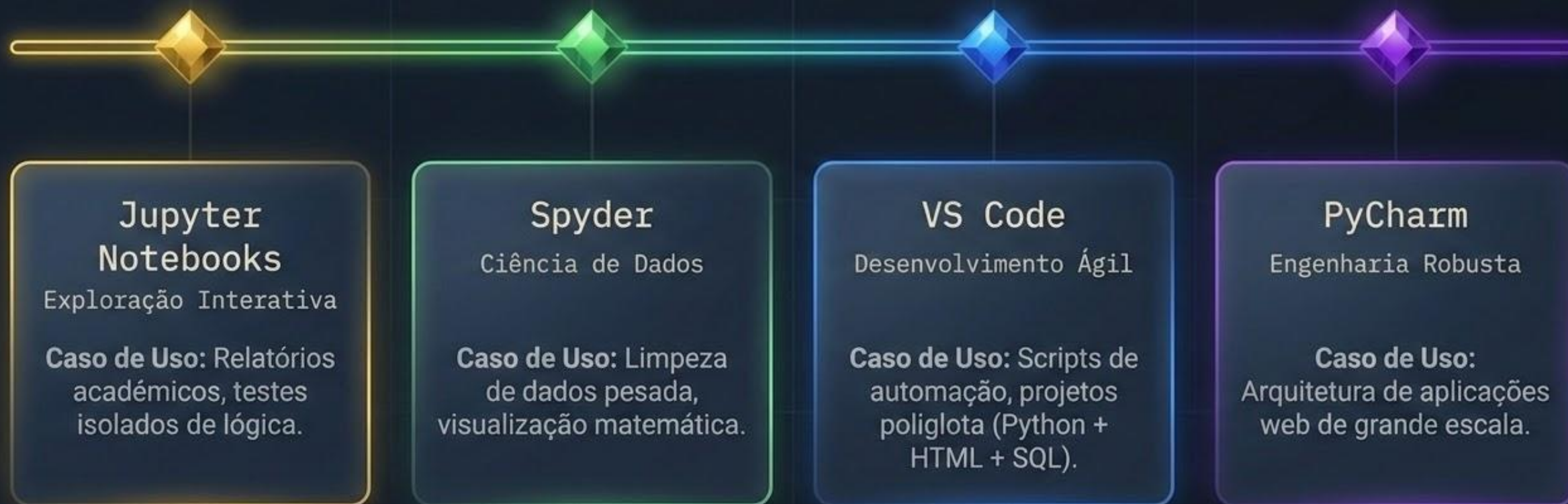
Permite mesclar blocos de código executáveis, visualizações ricas, gráficos interativos e textos explicativos formatados em um único documento prático.

Spyder

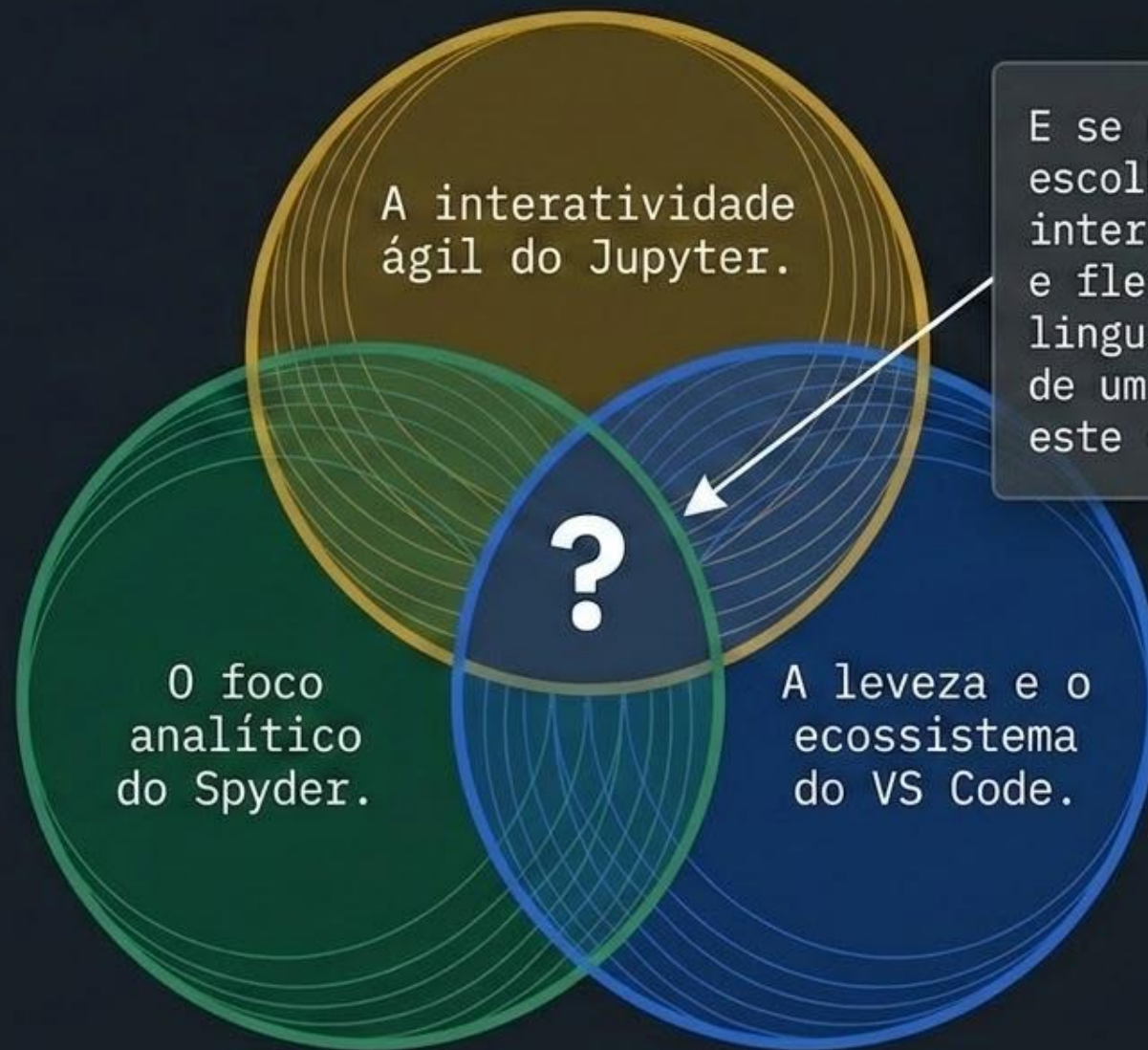
IDE científica desenhada especificamente por e para cientistas, engenheiros e analistas de dados.

Traz uma interface integrada e intuitiva que destaca o explorador de variáveis, painéis de documentação e gráficos avançados gerados em tempo real.

O Espectro de Complexidade dos Projetos



O Dilema do Estudante de Dados

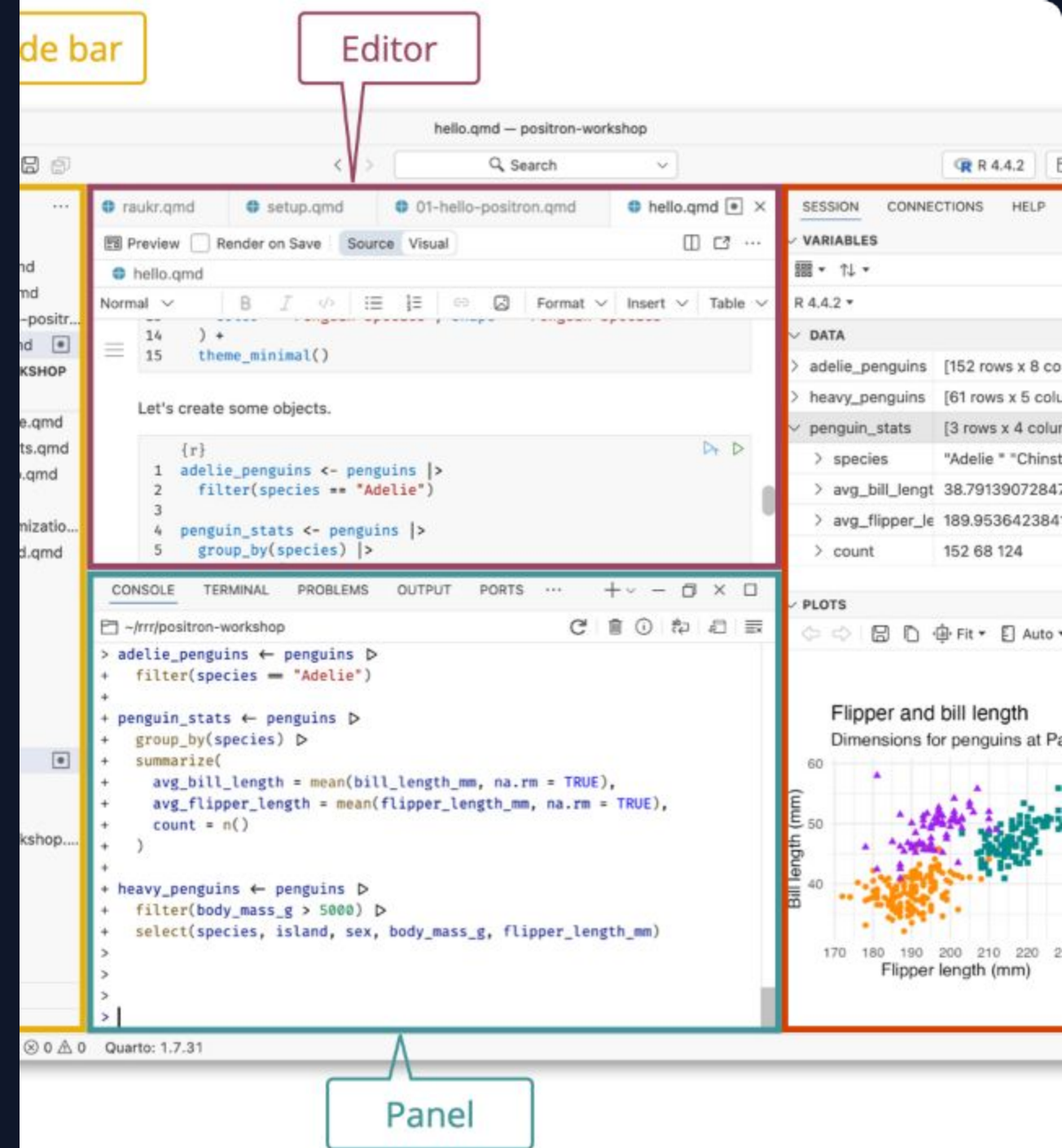


E se não precisássemos escolher entre desempenho, interface visual de dados e flexibilidade de linguagem? Precisamos de um elo definitivo para este curso.

Nossa escolha: Positron

Criado pela Posit, o Positron é focado em **Ciência de Dados**. Ele combina a velocidade do VS Code com as melhores ferramentas de visualização do RStudio.

Por que aqui? Visualização de variáveis em tempo real e explorador de DataFrames nativo.



Como baixar o Positron

Siga o passo a passo simples para obter a versão oficial.

1

Acesse o site oficial.

Acesse:

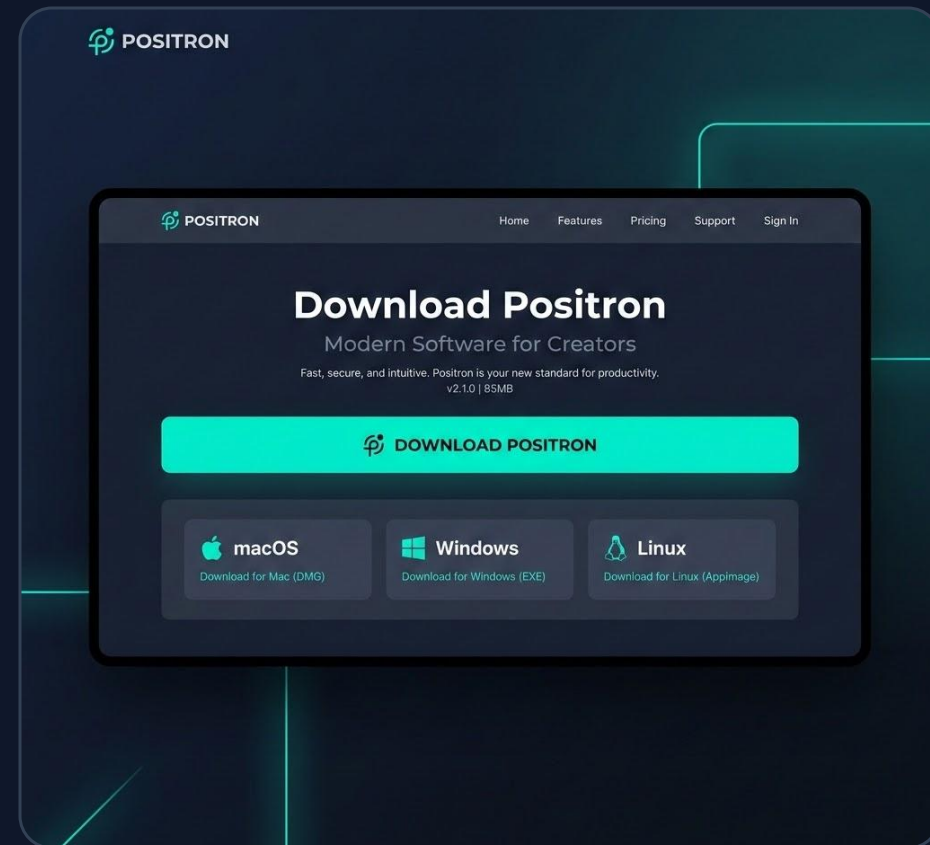
<https://positron.posit.co/>.

Procure a opção
"Download" e procure a
versão certa para o seu
sistema operacional

2

Instalação

Siga o passo a passo
oferecido pela aplicação
e espere a finalização do
download.



Gerenciamento de Pacotes

Expandindo as funcionalidades com o ecossistema PyPI.

| Entendendo o PIP

O **PIP** (Package Installer for Python) é o sistema de gerenciamento de pacotes padrão do Python.

Ele permite baixar bibliotecas prontas como *Pandas*, *Numpy* e *Scikit-Learn* do repositório **PyPI**.

```
# Instalando uma biblioteca
```

```
pip install pandas
```

```
# Atualizando o pip
```

```
python -m pip install --upgrade pip
```

Fundamentos do Python

Sintaxe, Variáveis e Estruturas de Dados.

| Conteúdo Programático

01 Tipos de Dados
int, float, str e bool

02 Listas & Tuplas
Estruturas
ordenadas

03 Dicionários
Estrutura
Chave-Valor

04 Loops & Condicionais
if, else, for e while

05 Funções
def, retornos e
docstrings

06 Leitura de CSV
Módulo nativo
(built-in)

| Dado: O Bloco Fundamental

Em Python, um dado representa a unidade fundamental de informação processada por um programa. Todo elemento é tratado internamente como um objeto com tipo, identidade e valor bem definidos.

Abordagem Estruturada

Esses blocos organizam a informação e podem variar desde valores numéricos simples até estruturas coletores altamente dinâmicas.

</> Identificação de Tipos

A função interna `type()` permite inspecionar de forma embutida e imediata a classificação de qualquer objeto no ecossistema Python.

```
print(type("nome do dado" ))
```

| Tipos de Dados Primitivos

int

Inteiros sem ponto decimal.

```
versao = 3
```

float

Valores com casas decimais.

```
nota = 95
```

str

Texto delimitado por aspas.

```
nome = 'Python'
```

bool

Lógica booleana.

```
ativo = True
```

| Prática em Aula

Exercício 1

Escreva variáveis para armazenar as seguintes informações sobre um produto específico:

 **Nome:** "NVIDIA GeForce RTX 5090" (texto)

 **Versão:** 32 (inteiro)

 **Preço:** 10.871,36 (decimal)

 **Estoque:** True (booleano)

Exiba no terminal o valor e o respectivo tipo usando `print()` e `type()`.

Estruturas de Dados: Listas

As **Listas** são coleções dinâmicas estruturadas ordenadamente que permitem modificações ao longo da execução.

- ✓ Criadas de forma direta através de colchetes [].
- ✓ Suportam índices numéricos baseados em zero.
- ✓ Permitem adição de tipos de dados heterogêneos.

Mapeamento de Índices

Índice	Elemento	Tipo
0	"Lua"	str
1	510	int
2	5.25	float

Manipulação de Listas: Métodos

Método	Ação Prática
<code>.append(v)</code>	Insere um item no final da lista
<code>.remove(v)</code>	Exclui o primeiro elemento correspondente
<code>.sort()</code>	Ordena os dados in-place
<code>len()</code>	Função que calcula a quantidade de itens

```
frutas = ["maçã", "banana"]  
frutas.append("uva")  
frutas.remove("banana")  
frutas.sort()  
tamanho = len(frutas)
```

Resultado: ['maçã', 'uva']

| Estruturas de Dados: Tuplas

Diferente das listas, as **Tuplas** são imutáveis. Uma vez instanciadas, seu conteúdo não pode ser reatribuído ou redimensionado.

- 🔒 Declaradas com o uso de parênteses ().
- 🔒 Excelente integridade para representar nomes de colunas ou configurações fixas.
- 🔒 Sem suporte a métodos de alteração (append/remove).

Desempacotamento Simples

Atribua múltiplos valores a variáveis de forma unificada.

```
coordenadas = (10, 20, 30)
x, y, z = coordenadas
```

| O Operador de Desempacotamento *

Como funciona

O caractere asterisco (*) captura múltiplos elementos remanescentes não declarados em variáveis explícitas, agrupando-os automaticamente em uma nova lista secundária.

```
notas = (10, 8, 5, 9, 7)

primeira, segunda, *resto = notas

print(primeira) # 10
print(resto) # [5, 9, 7]
```

| Prática em Aula

Exercício 2

Imagine que você está preparando o seu ambiente de programação e organizando as bibliotecas que usará para manipulação e visualização de dados.

- 1 Crie uma lista chamada `pacotes` com os valores:
`"tidyverse", "plotly", "gtsummary"`
- 2 Adicione o pacote `"shiny"` ao final da lista.
- 3 Remova o pacote `"plotly"`.
- 4 Ordene a lista em ordem alfabética.

| Dicionários: Mapeamento de Chaves

Os **Dicionários** são estruturados por pares de chave-valor. Funcionam de maneira idêntica a agendas ou catálogos indexados.

- ✓ Instanciados através de chaves curly { }.
- ✓ Chaves atuam como indexadores rápidos de conteúdo.
- ✓ Podem crescer de forma flexível e mutável durante a execução.

```
produto = {  
    "nome": "Notebook",  
    "preco": 4500.00,  
    "estoque": 10  
}
```

Acesso Seguro a Dicionários

.keys()

Captura exclusivamente o conjunto de chaves presentes.

.values()

Extraí de forma sequencial todos os valores armazenados.

.items()

Retorna os pares completos formatados em tuplas.

.get()

Método de busca segura que evita paradas do interpretador.

```
valor = produto.get("marca", "Não cadastrada") # Retorna padrão sem quebrar o código
```

| Prática em Aula

Exercício 3

Represente dados de uma disciplina de um sistema acadêmico.

-  Use chaves para "nome", "vagas" e "professor".
-  Imprima separadamente o retorno de `.keys()`.
-  Imprima o retorno correspondente a `.values()`.
-  Execute um teste seguro de busca por "sala" usando `.get()`.

Desafio Final - Frota Logística 🏆

Você foi contratado para criar o sistema interno de um aplicativo de entregas. Sua missão é organizar as informações dos veículos disponíveis utilizando as estruturas de dados adequadas:

- **1. Última Rota (Tupla):** Crie uma tupla imutável para guardar as coordenadas geográficas (Latitude e Longitude) do último ponto de entrega do veículo (ex: `(-7.115, -34.863)`).
- **2. Ficha do Veículo (Dicionário):** Cada veículo deve ser representado por um dicionário contendo as chaves: modelo, ano, capacidade_kg, em_operacao e ultima_coordenada.
- **3. Frota Ativa (Lista):** Agrupe os dicionários de pelo menos dois veículos dentro de uma única lista chamada `frota_aplicativo`.

📋 Estrutura Esperada do Dicionário

```
veiculo = {  
    "modelo": "Express Cargo", # str  
    "ano": 2024, # int  
    "capacidade_kg": 1200.50, # float  
    "em_operacao": True, # bool  
    "ultima_coordenada": (-7.115, -34.863) # tupla  
}
```

💡 Dica: Utilize a lista para gerenciar os múltiplos dicionários criados e simular o controle de frota em tempo real.

| Controle de Fluxo: Condicionais

Estruturas lógicas como o if/elif/else guiam o caminho de execução dependendo da veracidade de proposições booleanas.

Análogo do Semáforo

Avalia condições sequencialmente: Caso o sinal atenda ao critério **X**, realize a tarefa correspondente, caso contrário, prossiga para a próxima alternativa.

```
nota = 8

if nota ≥ 9:
    print("Excelente!")
elif nota ≥ 7:
    print("Aprovado.")
else:
    print("Reprovado.")
```

| Laços de Repetição: Estruturas

O Loop for

Ideal para iterações sobre objetos coletores cuja contagem de repetições é previamente conhecida.

```
frutas = ["maçã", "banana"]  
for f in frutas:  
    print(f)
```

O Loop while

Roda continuamente de acordo com critérios dinâmicos enquanto uma verificação booleana for True.

```
contador = 1  
while contador ≤ 3:  
    contador += 1
```

Prática em Aula

Exercício 4

Construa um analisador automático para um conjunto sequencial de notas:

```
notas = [9.5, 6.0, 8.2, 4.5, 7.0]
```

-  Utilize o laço de repetição for para ler as notas.
-  Valide notas maiores ou iguais a 9 como "Excelente!".
-  Valide notas maiores ou iguais a 7 como "Aprovado!".
-  Notas menores que 7 representam "Reprovado!".

| Modularização com Funções

Funções encapsulam trechos de lógica isolados para possibilitar manutenções limpas e total reutilização de rotinas dentro do programa.

- </> Definidas com a instrução reservada `def`.
- Recebem entradas denominadas **Parâmetros**.
- Retornam valores explícitos usando `return`.

Sintaxe de Escrita

```
def nome_funcao(parametro):  
    # Lógica de processamento  
    return resultado
```

| Função Prática com Parâmetros Padrão

É possível parametrizar valores iniciais que assumirão execução automática quando nenhum valor externo for alimentado à chamada.

O que é Docstring?

Strings triplas explicativas estruturadas logo abaixo da assinatura que atuam como documentação embarcada direta para ferramentas e programadores.

```
def calcular_desconto(preco, percentual=10):  
    """Calcula o desconto de um item."""  
    desconto = preco * (percentual / 100)  
    return preco - desconto  
  
final = calcular_desconto(100) # desconto padrão 10%
```

Prática em Aula

Exercício 5

Escreva o sistema de processamento de alertas analíticos de telemetria:

- ✓ Crie a função `analisar_leitura`.
- i Documente o método utilizando uma Docstring coerente.
- ⚙ Use o limite de leitura padrão de 100 unidades.
- ⚠ Se excedido, retorne "Alerta", senão retorne "Normal".